RELATÓRIO DE DESENVOLVIMENTO
Projeto Rei-Julian
Samuel Starke

1. INTRODUÇÃO

O projeto Rei-Julian foi desenvolvido com o objetivo de criar uma aplicação web capaz de calcular rotas seguras para ciclistas utilizando dados reais da cidade de Pelotas/RS.

Inicialmente, o sistema era apenas um MVP com um mapa fictício simples e poucas funcionalidades. Durante o desenvolvimento, o projeto evoluiu para uma aplicação funcional baseada em dados reais do OpenStreetMap, incorporando recursos como roteamento inteligente, tracking, simulação de deslocamento, zoom interativo, busca de ruas e comparação de rotas.

O foco principal do projeto foi utilizar conceitos de grafos, APIs HTTP, algoritmos de roteamento e visualização interativa para criar uma solução voltada à mobilidade urbana de ciclistas.

2. OBJETIVOS

2.1 Objetivo Geral

Desenvolver um sistema de rotas seguras para ciclistas utilizando dados reais da cidade de Pelotas/RS.

2.2 Objetivos Específicos

- Implementar cálculo de rotas baseado em grafos;
- Utilizar dados reais do OpenStreetMap;
- Diferenciar tipos de vias e ciclovias;
- Criar diferentes perfis de rota;
- Implementar tracking e recalculo automático;
- Desenvolver visualização interativa do mapa;
- Permitir simulação de deslocamento;
- Melhorar desempenho visual conforme o zoom.

3. TECNOLOGIAS UTILIZADAS

As principais tecnologias utilizadas no projeto foram Go para o backend e API HTTP, JavaScript para o frontend, HTML5 Canvas para renderização do mapa, OpenStreetMap como fonte de dados geográficos, Python para conversão dos dados OSM, JSON para representação do grafo e o algoritmo de Dijkstra para cálculo das rotas.

4. DESENVOLVIMENTO DO PROJETO

4.1 Estado Inicial

O projeto inicialmente possuía um mapa fictício simples, poucas ruas, roteamento básico, interface simples e ausência de funcionalidades como GPS, busca de locais e tracking completo.

A arquitetura inicial era composta pelos módulos cmd/api, internal/domain, internal/mapdata, internal/routing, internal/tracking, internal/transport/httpapi, web e maps.

4.2 Integração com OpenStreetMap

A principal evolução do projeto foi a substituição do mapa fictício por um mapa baseado em dados reais da cidade de Pelotas/RS utilizando OpenStreetMap.

Foi desenvolvido o script tools/osm_to_reijulian.py, responsável por realizar a leitura de arquivos XML do OpenStreetMap, extração de ruas e conexões, classificação de vias, identificação de ciclovias, cálculo de distância entre pontos e geração do grafo utilizado pelo sistema.

O mapa final passou a possuir aproximadamente 19.780 nós e 23.316 conexões, incluindo ciclovias, ciclofaixas, avenidas, ruas residenciais e rodovias reais.

4.3 Sistema de Roteamento

O sistema implementa cálculo de rotas utilizando o algoritmo de Dijkstra.

Foram implementados três perfis de rota: fastest, balanced e safest.

Cada perfil considera fatores como distância, segurança, tipo da via, infraestrutura cicloviária e risco estimado.

4.4 Sessões por Usuário

Inicialmente, o sistema utilizava apenas uma rota global em memória, causando conflito entre usuários.

Foi implementado suporte a session_id, permitindo múltiplas sessões independentes, tracking separado, simulações individuais e rotas independentes para cada usuário.

4.5 Busca de Ruas e Locais

Foi implementado o endpoint GET /search?q=..., permitindo localizar ruas e pontos do mapa através de nomes, removendo acentos e filtrando nomes genéricos.

4.6 Tracking e Recalculo

O sistema implementa tracking do usuário em relação à rota planejada.

O fluxo funciona da seguinte forma: o usuário calcula uma rota, o frontend envia a posição atual, o backend verifica desvios e, caso necessário, uma nova rota é calculada automaticamente.

4.7 Simulação de Movimento

Foi criado o módulo internal/simulation, responsável por simular o deslocamento do usuário ao longo da rota.

Também foram implementados os endpoints POST /simulation/start e POST /simulation/step.

4.8 Interface Interativa

A interface web passou a possuir zoom com roda do mouse, pan/arraste do mapa, seleção de pontos por clique, comparação de rotas, busca de locais, visualização de rotas, simulação visual e suporte inicial a GPS.

Também foi implementada otimização visual baseada no nível de zoom. Em zoom distante o sistema mostra apenas vias principais; em zoom intermediário mostra ruas secundárias; e em zoom aproximado mostra detalhes completos.

5. ENDPOINTS IMPLEMENTADOS

Os principais endpoints implementados foram:

GET /health
GET /map
GET /nodes
GET /stats
GET /search
POST /routes
POST /routes/compare
POST /tracking/update
POST /simulation/start
POST /simulation/step

6. TESTES E VALIDAÇÃO

Foram executados testes automatizados utilizando o comando go test ./...

Também foram realizados testes manuais dos endpoints HTTP e da interface gráfica.

O frontend JavaScript foi validado utilizando node --check web/app.js.

Todos os testes executaram corretamente.

7. LIMITAÇÕES ATUAIS

Apesar da evolução do sistema, ainda existem limitações como ausência de banco de dados, sessões armazenadas apenas em memória, busca geográfica simplificada, ausência de tráfego em tempo real, uso de Dijkstra em vez de A* e GPS ainda inicial.

8. MELHORIAS FUTURAS

As principais melhorias planejadas incluem integração com SQLite ou PostgreSQL, implementação de A*, tracking contínuo com watchPosition, exportação GPX/GeoJSON, cache de renderização, avaliações colaborativas de segurança e integração com dados urbanos em tempo real.

9. CONCLUSÃO

O projeto Rei-Julian evoluiu de um MVP simples para uma aplicação funcional baseada em dados reais do OpenStreetMap.

O sistema atualmente oferece roteamento seguro para ciclistas, múltiplos perfis de rota, tracking e recalculo, simulação de deslocamento, busca de locais, sessões independentes e visualização interativa do mapa.

O projeto permitiu aplicar conceitos de grafos, algoritmos de roteamento, APIs HTTP, processamento de dados geográficos, visualização interativa e otimização de interface.

Além disso, a utilização de dados reais aproximou o sistema de aplicações utilizadas em cenários urbanos reais.

METODOLOGIA DE DESENVOLVIMENTO E USO DE IA

O desenvolvimento do projeto Rei-Julian foi realizado com o uso de Inteligência Artificial integrada ao ambiente de trabalho por meio do Codex, operando diretamente dentro da estrutura de pastas do projeto.

A Inteligência Artificial foi responsável pela criação completa do sistema, incluindo a geração de diretórios, organização da arquitetura, implementação de código, criação de módulos, desenvolvimento da API, construção do frontend e integração dos diferentes componentes do sistema.

A arquitetura inicial do projeto, assim como a ideia central e os objetivos da aplicação, foram definidos pelo autor, que solicitou e aprovou a estrutura proposta. A

partir dessa base conceitual, toda a implementação técnica foi executada pela IA dentro do ambiente de desenvolvimento.

Isso inclui a criação dos seguintes elementos:

- estrutura completa de pastas do projeto;
- implementação do backend em Go;
- criação da API HTTP e seus endpoints;
- desenvolvimento do sistema de roteamento baseado em grafos;
- implementação do sistema de tracking e recálculo de rotas;
- criação do sistema de simulação de deslocamento;
- integração com dados reais do OpenStreetMap;
- desenvolvimento do conversor de dados OSM para o formato do projeto;
- construção da interface web utilizando HTML5 Canvas e JavaScript;
- implementação de zoom, pan e otimizações de renderização;
- criação de sistema de busca de ruas e locais;
- implementação de sessões independentes por usuário;
- criação e organização de testes automatizados.

Todo o processo de evolução do sistema ocorreu de forma iterativa dentro do ambiente de trabalho, com a IA executando diretamente as mudanças estruturais e funcionais no código-fonte conforme as solicitações definidas ao longo do desenvolvimento.

As decisões de escopo, objetivos e direção do projeto foram definidas pelo autor, enquanto a execução técnica completa, incluindo arquitetura detalhada, implementação e organização do código, foi realizada pela Inteligência Artificial no ambiente integrado do projeto.